In un sistema informatico che gestisce una piattaforma di ride-sharing, è necessario implementare in modo efficiente:

1. la ricerca rapida di un autista disponibile "più vicino" a un nuovo utente che richiede la corsa;
2. l'inserimento, rimozione e aggiornamento dinamico degli autisti disponibili man mano che essi si muovono o terminano una corsa;
3. la gestione di una lista di richieste in attesa quando nessun autista è immediatamente disponibile.

Quali **strutture dati** e **algoritmi** tratteresti nel corso di Algoritmi e Strutture Dati per progettare una soluzione efficiente a questo problema?

In particolare:

- indica quali strutture dati useresti per mantenere e aggiornare l'insieme degli autisti disponibili;
- spiega come gestiresti le richieste in attesa;
- descrivi il tipo di algoritmo utilizzato per assegnare un autista a una richiesta (ad esempio un approccio greedy o basato sulla ricerca in strutture bilanciate);
- analizza la complessità delle operazioni fondamentali (inserimento, ricerca, aggiornamento, estrazione di priorità);
- confronta brevemente la tua soluzione con almeno un approccio alternativo meno efficiente.

Motiva ogni scelta alla luce degli argomenti studiati nel corso.

Una possibile soluzione, coerente con il programma del corso, è progettare il sistema combinando **strutture dati per dizionari**, **alberi bilanciati**, **code/heap** e un **algoritmo di assegnazione greedy**.

Questa scelta è perfettamente in linea con gli argomenti del programma del corso, che include hash table, heap, alberi di ricerca binari, AVL, alberi rosso-neri, code e tecniche greedy, oltre all'analisi della complessità.

1. Insieme degli autisti disponibili

Il problema richiede tre operazioni frequenti:

- inserire un autista quando diventa disponibile;
- rimuoverlo quando accetta una corsa;
- aggiornarne la posizione mentre si muove.

Dal punto di vista algoritmico, io userei una **soluzione a due livelli**.

a) Hash table per accesso diretto all'autista

Mantengo una **tabella hash** che associa driver_id -> record autista.

Questo permette:

- ricerca del record in tempo medio **O(1)**;
- aggiornamento rapido dei dati dell'autista;
- rimozione veloce conoscendo l'identificativo.

La hash table è la struttura giusta quando serve comportamento da dizionario con accesso quasi costante, ed è esplicitamente tra gli argomenti del corso.

b) Alberi bilanciati per organizzare gli autisti nello spazio

Per la ricerca del "più vicino", una semplice hash table non basta, perché non mantiene un ordinamento spaziale. Restando negli strumenti tipici di un corso di Algoritmi e Strutture Dati, userei una **suddivisione del territorio in zone/celle** e, per ogni zona, un **albero bilanciato** (AVL o rosso-nero) contenente gli autisti disponibili in quella zona. Gli alberi bilanciati sono presenti nel syllabus e garantiscono operazioni logaritmiche nel caso peggiore.

L'idea è questa:

- la città viene divisa in celle;
- ogni autista appartiene alla cella corrispondente alla sua posizione corrente;
- per ogni cella mantengo un insieme ordinato di autisti disponibili;

- quando un autista si sposta e cambia cella, lo rimuovo dall'albero della vecchia cella e lo inserisco in quello della nuova.

In questo modo evito di cercare il vicino su tutto l'insieme globale degli autisti.

Perché non un solo albero globale?

Un unico AVL ordinato, ad esempio, per latitudine o per una chiave lessicografica (x,y) non risolve bene il problema della vicinanza geometrica: l'ordine totale di un BST non coincide con la distanza euclidea o stradale. La discretizzazione in zone riduce questo limite e rende la soluzione compatibile con le strutture viste a lezione.

2. Gestione delle richieste in attesa

Quando nessun autista è disponibile, le richieste vanno memorizzate in una struttura separata.

Caso base: coda FIFO

Se il criterio è la sola equità temporale, uso una **coda FIFO**:

- inserimento richiesta in fondo: **O(1)**;
- estrazione della più vecchia: **O(1)**.

Questa è la scelta più naturale se il sistema vuole servire le richieste nell'ordine di arrivo.

Caso più realistico: priority queue con heap

Se invece voglio introdurre una priorità, ad esempio:

- orario di arrivo,
- classe di servizio,
- tempo di attesa già accumulato,

allora userei una **coda con priorità implementata con heap**. Anche questa struttura è prevista dal corso.

Con un heap ho:

- inserimento richiesta: **O(log r)**;
- estrazione della richiesta prioritaria: **O(log r)**;
- lettura della massima/minima priorità: **O(1)**,

dove r è il numero di richieste in attesa.

Dal punto di vista didattico, io direi che:

- **FIFO** è preferibile se si privilegia la semplicità e la fairness;
- **heap** è preferibile se il sistema deve ottimizzare il servizio secondo un criterio di priorità.

3. Algoritmo di assegnazione dell'autista

L'assegnazione può essere descritta come un **algoritmo greedy**: per ogni nuova richiesta si sceglie l'autista disponibile che, in quel momento, appare migliore secondo una funzione di costo, ad esempio la distanza o il tempo stimato di arrivo. L'approccio greedy è coerente con il programma del corso.

Strategia proposta

Dato un utente che richiede una corsa:

1. individuo la cella in cui si trova l'utente;
2. cerco autisti disponibili nella stessa cella;
3. se non ce ne sono, espando la ricerca alle celle adiacenti;
4. raccolgo i candidati trovati;
5. seleziono quello con distanza minima;
6. lo rimuovo dalla struttura degli autisti disponibili;
7. se non esistono candidati, inserisco la richiesta nella struttura delle attese.

Questa è una politica greedy perché compie una scelta locale immediata: assegna l'autista migliore tra quelli attualmente disponibili.

Variante su grafo

Se il territorio è modellato come **grafo stradale**, allora la nozione di "vicino" può essere basata sul cammino minimo. In tal caso, tra gli strumenti del corso, si può citare **Dijkstra**, usando una priority queue per esplorare il grafo dal nodo dell'utente fino a raggiungere un nodo occupato da un autista disponibile. È una variante più accurata, ma anche più costosa computazionalmente. Gli algoritmi su grafi e i cammini minimi fanno parte del syllabus.

4. Complessità delle operazioni fondamentali

Supponiamo di avere:

- n autisti disponibili totali;
- k autisti in una certa cella;

- r richieste in attesa.

Operazioni sugli autisti

Con hash table + alberi bilanciati per cella:

- **inserimento autista disponibile**:
    - hash table: **O(1)** medio
    - inserimento nell'AVL/RB tree della cella: **O(log k)**
    - totale: **O(log k)** dominante
- **rimozione autista**:
    - recupero tramite hash: **O(1)** medio
    - cancellazione dall'albero della cella: **O(log k)**
    - totale: **O(log k)**
- **aggiornamento posizione**:
    - se resta nella stessa cella: aggiornamento record **O(1)** medio
    - se cambia cella: rimozione da un albero e inserimento in un altro quindi **O(log k_1 + log k_2)**, che possiamo scrivere come **O(log n)** nel caso peggiore

Ricerca dell'autista più vicino

Se il candidato è cercato:

- prima nella stessa cella,
- poi in celle vicine,

la complessità dipende dal numero di celle esplorate e dal numero di candidati esaminati. In forma semplificata:

- accesso alla cella: **O(1)**;
- visita degli alberi delle celle coinvolte: circa **O(\sum log k_i)**;
- selezione del migliore tra m candidati: **O(m)**.

Nel caso medio, se la distribuzione geografica è ragionevolmente uniforme, la ricerca è molto più efficiente di una scansione globale. Nel caso peggiore, se quasi tutti gli autisti sono concentrati in poche celle o se devo espandere molto la ricerca, posso arrivare a costi vicini a **O(n)**.

Richieste in attesa

- con **coda FIFO**:
    - inserimento: **O(1)**
    - estrazione: **O(1)**
- con **heap**:
    - inserimento: **O(log r)**
    - estrazione prioritaria: **O(log r)**
    - lettura priorità massima/minima: **O(1)**

Questa analisi della complessità è esattamente il tipo di valutazione richiesto dal corso: confronto tra costi di inserimento, ricerca, aggiornamento ed estrazione di priorità con notazione asintotica.

5. Confronto con un approccio alternativo meno efficiente

L'approccio alternativo più semplice consiste nel mantenere tutti gli autisti disponibili in una **lista o array non ordinato**.

Soluzione ingenua

Per ogni richiesta:

- scorro tutti gli autisti disponibili;
- calcolo la distanza da ciascuno;
- scelgo il minimo.

Complessità:

- inserimento autista: **O(1)** in coda all'array/lista;
- rimozione autista: **O(n)** se devo cercarlo;
- ricerca del più vicino: **O(n)**;
- aggiornamento posizione: **O(1)** se conosco il record, ma resta costosa la successiva ricerca globale.

Perché è peggiore?

Questa soluzione è facile da implementare, ma non scala. Se il numero di autisti cresce, ogni nuova richiesta richiede una scansione lineare. In una piattaforma reale, con molte richieste simultanee, questo diventa rapidamente inefficiente.

La mia proposta, invece, introduce una struttura più ricca:

- **hash table** per accesso diretto,
- **alberi bilanciati** per aggiornamenti logaritmici,
- **coda o heap** per le richieste in attesa,
- **strategia greedy** per l'assegnazione immediata.

È quindi una soluzione molto più coerente con gli obiettivi del corso: scegliere strutture dati adeguate, motivarne l'uso e valutarne i compromessi in termini di complessità.

Conclusione

In una risposta d'esame ben formulata, sulla base degli argomenti del corso, è:

- **hash table** per identificare e aggiornare rapidamente gli autisti;
- **alberi bilanciati AVL o rosso-neri**, organizzati per zona, per mantenere l'insieme dinamico degli autisti disponibili;
- **coda FIFO** oppure **priority queue con heap** per le richieste in attesa;
- **algoritmo greedy** di assegnazione, che sceglie il miglior autista disponibile localmente;
- eventuale riferimento a **Dijkstra** se la vicinanza va calcolata sulla rete stradale anziché in modo geometrico.

Questa soluzione è preferibile a una semplice lista lineare perché migliora nettamente i costi di aggiornamento e, soprattutto, la ricerca dell'autista da assegnare.

Alcuni **frammenti di codice Python**, da accompagnare una possibile implementazione coerente con la soluzione proposta (hash table + strutture ordinate + coda/heap + strategia greedy).

1. Struttura dati per gli autisti (hash table)

Useremo un dizionario Python (hash table) per accedere rapidamente agli autisti.

```
# Definizione di una classe per rappresentare un autista
class Autista:
    def __init__(self, driver_id, x, y):
        self.id = driver_id     # identificativo univoco
        self.x = x              # coordinata x (es. longitudine)
        self.y = y              # coordinata y (es. latitudine)

    def posizione(self):
        return (self.x, self.y)
# Dizionario globale: driver_id -> oggetto Autista
autisti = {}

def aggiungi_autista(driver_id, x, y):
    """Inserisce un nuovo autista disponibile"""
    autisti[driver_id] = Autista(driver_id, x, y)

def rimuovi_autista(driver_id):
    """Rimuove un autista (es. quando accetta una corsa)"""
    if driver_id in autisti:
        del autisti[driver_id]

def aggiorna_posizione(driver_id, x, y):
    """Aggiorna la posizione di un autista"""
```

```
    if driver_id in autisti:
        autisti[driver_id].x = x
        autisti[driver_id].y = y
```

✔ Accesso medio: **O(1)**

2. Suddivisione in celle (semplificazione spaziale)

Simuliamo la suddivisione in zone con una griglia.

```
from collections import defaultdict

# Dizionario: cella -> insieme di driver_id
celle = defaultdict(set)

DIM_CELLA = 10  # dimensione della cella (semplificata)

def calcola_cella(x, y):
    """Calcola la cella a cui appartiene una posizione"""
    return (x // DIM_CELLA, y // DIM_CELLA)
def inserisci_in_cella(driver_id):
    """Inserisce l'autista nella cella corretta"""
    autista = autisti[driver_id]
    cella = calcola_cella(autista.x, autista.y)
    celle[cella].add(driver_id)
def rimuovi_da_cella(driver_id):
    """Rimuove l'autista dalla cella corrente"""
    autista = autisti[driver_id]
    cella = calcola_cella(autista.x, autista.y)
    celle[cella].discard(driver_id)
def aggiorna_cella(driver_id, new_x, new_y):
```

```
    """Aggiorna la cella quando l'autista si sposta"""
    autista = autisti[driver_id]

    vecchia_cella = calcola_cella(autista.x, autista.y)
    nuova_cella = calcola_cella(new_x, new_y)

    if vecchia_cella != nuova_cella:
        celle[vecchia_cella].discard(driver_id)
        celle[nuova_cella].add(driver_id)

    # aggiorna posizione
    autista.x = new_x
    autista.y = new_y
```

✔ Operazioni efficienti (quasi costanti)

3. Funzione di distanza

```
import math

def distanza(a, b):
    """Calcola la distanza euclidea tra due punti"""
    return math.sqrt((a.x - b[0])**2 + (a.y - b[1])**2)
```

4. Ricerca greedy dell'autista più vicino

```
def trova_autista_piu_vicino(x, y):
    """Trova l'autista più vicino usando una strategia greedy"""

    cella_utente = calcola_cella(x, y)
    candidati = []
```

```
    # Consideriamo la cella corrente e quelle adiacenti
    for dx in [-1, 0, 1]:
        for dy in [-1, 0, 1]:
            cella = (cella_utente[0] + dx, cella_utente[1] + dy)

            for driver_id in celle[cella]:
                candidati.append(autisti[driver_id])

    # Se non ci sono autisti disponibili
    if not candidati:
        return None

    # Scelta greedy: minimo della distanza
    migliore = min(candidati, key=lambda a: distanza(a, (x, y)))

    return migliore
```

✔ Strategia: **greedy (minimizza distanza locale)**

5. Assegnazione dell'autista

```
def assegna_corsa(x, y):
    """Assegna un autista a una richiesta"""

    autista = trova_autista_piu_vicino(x, y)

    if autista is None:
        print("Nessun autista disponibile")
        return None
```

```
    # Rimuoviamo l'autista dalle strutture (non più disponibile)
    rimuovi_da_cella(autista.id)
    rimuovi_autista(autista.id)

    print(f"Assegnato autista {autista.id}")
    return autista
```

6. Gestione richieste in attesa (priority queue)

Usiamo un heap (coda con priorità).

```
import heapq
import time

# Heap di richieste: (priorità, timestamp, richiesta)
richieste = []

class Richiesta:
    def __init__(self, x, y):
        self.x = x
        self.y = y
        self.timestamp = time.time()
def aggiungi_richiesta(x, y):
    """Inserisce una richiesta in attesa"""

    richiesta = Richiesta(x, y)

    # Priorità: timestamp (FIFO)
    heapq.heappush(richieste, (richiesta.timestamp, richiesta))
def processa_richieste():
```

```
    """Prova ad assegnare autisti alle richieste in attesa"""

    while richieste:
        _, richiesta = heapq.heappop(richieste)

        autista = assegna_corsa(richiesta.x, richiesta.y)

        if autista is None:
            # reinseriamo la richiesta se non soddisfatta
            heapq.heappush(richieste, (richiesta.timestamp, richiesta))
            break
```

✔ Complessità heap:

- inserimento: **O(log r)**
- estrazione: **O(log r)**

🔹 **7. Esempio d'uso**

```
# Inseriamo alcuni autisti
aggiungi_autista(1, 10, 10)
aggiungi_autista(2, 15, 15)
aggiungi_autista(3, 50, 50)

# Inserimento nelle celle
for driver_id in autisti:
    inserisci_in_cella(driver_id)

# Richiesta di corsa
assegna_corsa(12, 12)
```

Caso senza autisti

aggiungi_richiesta(100, 100)

processa_richieste()

✅ Conclusione didattica

Questi frammenti mostrano chiaramente:

- uso di **hash table (dict)** → accesso O(1)
- uso di **strutturazione spaziale** → riduzione della ricerca
- uso di **heap (priority queue)** → gestione richieste
- uso di **algoritmo greedy** → scelta del driver